```
import os, base64, subprocess, tempfile, re, json, textwrap
from flask import Flask, request, jsonify, send_file, render_template
from werkzeug.utils import secure_filename

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = 100 * 1024 * 1024

ANTHROPIC_KEY = os.environ.get("ANTHROPIC_API_KEY", "")

# ── Image extraction ──────────────────────────────────────────────

def extract_images(pdf_path, out_dir):
    prefix = os.path.join(out_dir, "img")
    subprocess.run(["pdfimages", "-j", pdf_path, prefix],
                   check=True, capture_output=True)
    imgs = sorted(
        [os.path.join(out_dir, f) for f in os.listdir(out_dir)
         if re.match(r"img-\d+\.jpe?g$", f, re.IGNORECASE)],
        key=lambda p: int(re.search(r"(\d+)", os.path.basename(p)).group(1))
    )
    return imgs

def extract_meta(pdf_path):
    try:
        result = subprocess.run(
            ["pdftotext", "-layout", pdf_path, "-"],
            capture_output=True, text=True, check=True
        )
        text = result.stdout
    except Exception:
        return {"project": "Project", "address": "", "date": ""}

    meta = {"project": "", "address": "", "date": ""}
    m = re.search(r"Before\s*&\s*After\s+([^\n]{3,60})", text, re.IGNORECASE)
    if m:
        meta["address"] = m.group(1).strip()
    m = re.search(
        r"(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\s+\d{1,2},?\s+\
        text, re.IGNORECASE
    )
    if m:
        meta["date"] = m.group(0)
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    for i, line in enumerate(lines):
```

```
        if "formo" in line.lower() and i > 0:
            c = lines[i - 1]
            if len(c) < 50 and not re.search(r"\d{4}|before|after|renovation", c,
                meta["project"] = c
                break
    return meta

# ── AI captions ──────────────────────────────────────────────────────────

def ai_describe(img_path, num):
    if not ANTHROPIC_KEY:
        return {"section": "General", "title": f"Photo {num}",
                "description": "", "tags": []}
    import anthropic
    client = anthropic.Anthropic(api_key=ANTHROPIC_KEY)
    with open(img_path, "rb") as f:
        b64 = base64.b64encode(f.read()).decode()
    try:
        msg = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=180,
            messages=[{"role": "user", "content": [
                {"type": "image",
                 "source": {"type": "base64", "media_type": "image/jpeg", "data":
                {"type": "text", "text":
                    f'Before/after renovation photo #{num}. BEFORE=left, AFTER=ri
                    f'Reply ONLY valid JSON:\n'
                    f'{{"section":"<Exterior|Deck|Doors|Flooring|Bathrooms|Kitche
                    f'Paint & Drywall|Windows & Lighting|Trim & Molding|Utilities
                    f'"title":"<4-6 word English title>",'
                    f'"description":"<EXACTLY 2 short English sentences. '
                    f'First: problem before. Second: what was done. Max 18 words
                    f'"tags":["<tag1>","<tag2>"]}}'}
            ]}]
        )
        text = msg.content[0].text.strip().replace("```json", "").replace("```",
        return json.loads(text)
    except Exception:
        return {"section": "General", "title": f"Photo {num}",
                "description": "Renovation work completed.", "tags": []}

# ── PDF builder using fpdf2 ──────────────────────────────────────────────

GOLD  = (201, 169, 110)
BLACK = (25,  24,  22)
WHITE = (248, 244, 239)
GRAY  = (197, 188, 176)
```

```
DARK  = (37,  34,  32)
MUTED = (122, 112, 96)

PW, PH = 215.9, 279.4   # Letter in mm
MARGIN = 14


def hex_rgb(h):
    h = h.lstrip("#")
    return tuple(int(h[i:i+2], 16) for i in (0, 2, 4))


def build_pdf(meta, cover_path, photo_imgs, captions, out_path):
    from fpdf import FPDF
    from PIL import Image

    project = meta.get("project", "")
    address = meta.get("address", "")
    date    = meta.get("date", "")

    pdf = FPDF(orientation="P", unit="mm", format="Letter")
    pdf.set_auto_page_break(False)

    # ── COVER PAGE ──────────────────────────────────────────────────────
    pdf.add_page()

    # Dark background
    pdf.set_fill_color(*BLACK)
    pdf.rect(0, 0, PW, PH, "F")

    # Cover photo with dark overlay
    try:
        img = Image.open(cover_path).convert("RGB")
        w, h = img.size
        scale = PW / w
        ph_img = h * scale
        # Darken it
        from PIL import ImageEnhance
        dark_img = ImageEnhance.Brightness(img).enhance(0.28)
        tmp_cover = cover_path + "_dark.jpg"
        dark_img.save(tmp_cover, "JPEG", quality=80)
        pdf.image(tmp_cover, x=0, y=0, w=PW, h=min(ph_img, PH))
    except Exception:
        pass

    # Gradient overlay (simulate with rects)
    for i in range(20):
```

```
    alpha = int(180 * (i / 19))
    y_pos = PH * 0.3 + (PH * 0.7) * (i / 19)
    pdf.set_fill_color(25, 24, 22)
    pdf.set_alpha(i / 19 * 0.95)
    pdf.rect(0, y_pos, PW, PH * 0.04, "F")
pdf.set_alpha(1)

# Bottom solid block
pdf.set_fill_color(*BLACK)
pdf.rect(0, PH * 0.65, PW, PH * 0.35, "F")

# Gold line
pdf.set_draw_color(*GOLD)
pdf.set_line_width(0.4)
pdf.line(PW/2 - 20, PH * 0.52, PW/2 + 20, PH * 0.52)

# FORMO
pdf.set_font("Helvetica", "B", 22)
pdf.set_text_color(*WHITE)
pdf.set_xy(0, PH * 0.54)
pdf.cell(PW, 10, "FORMO", align="C")

# RENOVATION
pdf.set_font("Helvetica", "", 9)
pdf.set_text_color(*GOLD)
pdf.set_xy(0, PH * 0.59)
pdf.cell(PW, 6, "RENOVATION", align="C")

# Before & After
pdf.set_font("Helvetica", "B", 36)
pdf.set_text_color(*WHITE)
pdf.set_xy(0, PH * 0.64)
pdf.cell(PW, 16, "Before & After", align="C")

# Address line
pdf.set_font("Helvetica", "", 10)
pdf.set_text_color(*GRAY)
addr_line = f"{address}  ·  {project}  ·  {date}"
pdf.set_xy(0, PH * 0.73)
pdf.cell(PW, 7, addr_line, align="C")

# Stats
stats = [
    (str(len(photo_imgs)), "PHOTOS"),
    (str(len(set(c.get("section","General") for c in captions))), "WORK AREAS
    ("100%", "BEFORE & AFTER"),
]
```

```
col_w = PW / 3
for i, (val, lbl) in enumerate(stats):
    x = i * col_w
    pdf.set_font("Helvetica", "B", 24)
    pdf.set_text_color(*GOLD)
    pdf.set_xy(x, PH * 0.80)
    pdf.cell(col_w, 12, val, align="C")
    pdf.set_font("Helvetica", "", 7)
    pdf.set_text_color(*MUTED)
    pdf.set_xy(x, PH * 0.86)
    pdf.cell(col_w, 5, lbl, align="C")

# Tagline
pdf.set_font("Helvetica", "", 8)
pdf.set_text_color(*MUTED)
pdf.set_xy(0, PH * 0.91)
pdf.cell(PW, 6, "BUILT ON QUALITY  ·  DRIVEN BY INTEGRITY", align="C")

# ── GOLD BAND PAGE ──────────────────────────────────────────────────
pdf.add_page()
pdf.set_fill_color(*BLACK)
pdf.rect(0, 0, PW, PH, "F")

# Gold band
pdf.set_fill_color(*GOLD)
pdf.rect(0, 0, PW, 16, "F")
pdf.set_font("Helvetica", "B", 8)
pdf.set_text_color(*BLACK)
pdf.set_xy(MARGIN, 4)
pdf.cell(PW/2, 8, f"FULL RENOVATION REPORT  ·  {address.upper()}", align="L")
pdf.set_xy(PW/2, 4)
pdf.cell(PW/2 - MARGIN, 8, f"FORMO RENOVATION  ·  {date.upper()}", align="R")

# Group photos by section
ORDER = ["Exterior","Deck","Doors","Flooring","Bathrooms","Kitchen",
         "Paint & Drywall","Windows & Lighting","Trim & Molding","Utilities",
grouped = {}
for i, cap in enumerate(captions):
    sec = cap.get("section", "General")
    grouped.setdefault(sec, []).append((i, cap))
secs = [s for s in ORDER if s in grouped] + [s for s in grouped if s not in O

y_cursor = 22
sec_num = 0

for sec in secs:
    sec_num += 1
```

```
items = grouped[sec]

for idx, (photo_idx, cap) in enumerate(items):
    img_path = photo_imgs[photo_idx]
    title = cap.get("title", f"Photo {photo_idx+1}")
    desc  = cap.get("description", "")
    tags  = cap.get("tags", [])
    has_desc = bool(desc)
    card_h = 62 if has_desc else 52

    # New page if needed
    if y_cursor + card_h > PH - 20:
        # Footer on current page
        _add_footer(pdf, project, address, date)
        pdf.add_page()
        pdf.set_fill_color(*BLACK)
        pdf.rect(0, 0, PW, PH, "F")
        y_cursor = 14

    # Section header — only for first item in section
    if idx == 0:
        if y_cursor + 12 + card_h > PH - 20:
            _add_footer(pdf, project, address, date)
            pdf.add_page()
            pdf.set_fill_color(*BLACK)
            pdf.rect(0, 0, PW, PH, "F")
            y_cursor = 14

        # Section number + title
        pdf.set_font("Helvetica", "", 8)
        pdf.set_text_color(*GOLD)
        pdf.set_xy(MARGIN, y_cursor)
        pdf.cell(12, 6, f"{sec_num:02d}", align="L")
        pdf.set_font("Helvetica", "B", 11)
        pdf.set_text_color(*WHITE)
        pdf.set_xy(MARGIN + 12, y_cursor)
        pdf.cell(PW - MARGIN*2 - 12, 6, sec.upper(), align="L")

        # Gold underline
        pdf.set_draw_color(*GOLD)
        pdf.set_line_width(0.3)
        pdf.line(MARGIN, y_cursor + 7, PW - MARGIN, y_cursor + 7)
        y_cursor += 11

    # Card background
    pdf.set_fill_color(*DARK)
    pdf.rect(MARGIN, y_cursor, PW - MARGIN*2, card_h, "F")
```

```
# Photo (left half)
img_w = (PW - MARGIN*2) * 0.52
img_x = MARGIN
try:
    pdf.image(img_path, x=img_x, y=y_cursor, w=img_w, h=card_h)
except Exception:
    pass

# Photo badge
pdf.set_fill_color(*BLACK)
pdf.set_fill_color(25, 24, 22)
pdf.rect(img_x + 2, y_cursor + 2, 14, 5, "F")
pdf.set_font("Helvetica", "B", 6)
pdf.set_text_color(*GOLD)
pdf.set_xy(img_x + 2, y_cursor + 2.5)
pdf.cell(14, 4, f"#{photo_idx+1}", align="C")

# Text area (right half)
tx = MARGIN + img_w + 4
tw = PW - MARGIN - tx - 2
ty = y_cursor + 8

# Gold line accent
pdf.set_draw_color(*GOLD)
pdf.set_line_width(0.5)
pdf.line(tx, ty - 3, tx + 10, ty - 3)

# Title
pdf.set_font("Helvetica", "B", 9)
pdf.set_text_color(*WHITE)
# Wrap title
title_lines = textwrap.wrap(title.upper(), width=28)
for tl in title_lines[:2]:
    pdf.set_xy(tx, ty)
    pdf.cell(tw, 5, tl, align="L")
    ty += 5

ty += 2

# Description
if has_desc:
    pdf.set_font("Helvetica", "", 7.5)
    pdf.set_text_color(*GRAY)
    desc_lines = textwrap.wrap(desc, width=38)
    for dl in desc_lines[:4]:
        pdf.set_xy(tx, ty)
```

```
                pdf.cell(tw, 4.2, dl, align="L")
                ty += 4.2
            ty += 2

        # Tags
        tag_x = tx
        for tag in tags[:2]:
            tag_text = tag.upper()
            tag_w = min(len(tag_text) * 1.8 + 4, tw)
            pdf.set_draw_color(*GOLD)
            pdf.set_line_width(0.2)
            pdf.rect(tag_x, ty, tag_w, 5, "D")
            pdf.set_font("Helvetica", "", 5.5)
            pdf.set_text_color(*GOLD)
            pdf.set_xy(tag_x, ty + 0.5)
            pdf.cell(tag_w, 4, tag_text, align="C")
            tag_x += tag_w + 2

        # Section label bottom
        pdf.set_font("Helvetica", "", 6)
        pdf.set_text_color(*MUTED)
        pdf.set_xy(tx, y_cursor + card_h - 7)
        pdf.cell(tw, 5, sec.upper(), align="L")

        # Separator line
        pdf.set_draw_color(50, 46, 42)
        pdf.set_line_width(0.2)
        pdf.line(MARGIN, y_cursor + card_h, PW - MARGIN, y_cursor + card_h)

        y_cursor += card_h + 2

# ── SUMMARY PAGE ──────────────────────────────────────────────────
_add_footer(pdf, project, address, date)
pdf.add_page()
pdf.set_fill_color(*BLACK)
pdf.rect(0, 0, PW, PH, "F")

# Gold band
pdf.set_fill_color(*GOLD)
pdf.rect(0, 0, PW, 16, "F")
pdf.set_font("Helvetica", "B", 8)
pdf.set_text_color(*BLACK)
pdf.set_xy(MARGIN, 4)
pdf.cell(PW - MARGIN*2, 8, "PROJECT SUMMARY", align="C")

# Title
pdf.set_font("Helvetica", "B", 28)
```

```
pdf.set_text_color(*GOLD)
pdf.set_xy(0, 36)
pdf.cell(PW, 14, "Project Summary", align="C")

# KPI boxes
kpis = [
    (str(len(photo_imgs)), "Photos Documented"),
    (str(len(secs)), "Work Areas"),
    ("100%", "Scope Coverage"),
]
box_w = (PW - MARGIN*2 - 8) / 3
box_x = MARGIN
box_y = 60
for val, lbl in kpis:
    pdf.set_fill_color(*DARK)
    pdf.rect(box_x, box_y, box_w, 28, "F")
    pdf.set_draw_color(*GOLD)
    pdf.set_line_width(0.3)
    pdf.rect(box_x, box_y, box_w, 28, "D")
    pdf.set_font("Helvetica", "B", 26)
    pdf.set_text_color(*GOLD)
    pdf.set_xy(box_x, box_y + 4)
    pdf.cell(box_w, 14, val, align="C")
    pdf.set_font("Helvetica", "", 7)
    pdf.set_text_color(*MUTED)
    pdf.set_xy(box_x, box_y + 18)
    pdf.cell(box_w, 6, lbl.upper(), align="C")
    box_x += box_w + 4

# Scope table
ty = 104
pdf.set_font("Helvetica", "B", 8)
pdf.set_text_color(*GOLD)
pdf.set_xy(MARGIN, ty)
pdf.cell(PW - MARGIN*2, 7, "SCOPE OF WORK", align="L")
pdf.set_draw_color(*GOLD)
pdf.set_line_width(0.3)
pdf.line(MARGIN, ty + 8, PW - MARGIN, ty + 8)
ty += 12

col1 = 52
col2 = PW - MARGIN*2 - col1
for sec in secs:
    count = len(grouped[sec])
    pdf.set_fill_color(*DARK)
    pdf.rect(MARGIN, ty, PW - MARGIN*2, 9, "F")
    pdf.set_font("Helvetica", "B", 7)
```

```
            pdf.set_text_color(*GOLD)
            pdf.set_xy(MARGIN + 2, ty + 1.5)
            pdf.cell(col1, 6, sec.upper(), align="L")
            pdf.set_font("Helvetica", "", 7)
            pdf.set_text_color(*GRAY)
            pdf.set_xy(MARGIN + col1 + 2, ty + 1.5)
            pdf.cell(col2 - 4, 6, f"{count} before & after photo{'s' if count>1 else
            pdf.set_draw_color(50, 46, 42)
            pdf.set_line_width(0.15)
            pdf.line(MARGIN, ty + 9, PW - MARGIN, ty + 9)
            ty += 9
            if ty > PH - 30:
                break

    _add_footer(pdf, project, address, date)
    pdf.output(out_path)


def _add_footer(pdf, project, address, date):
    from fpdf import FPDF
    pdf.set_fill_color(37, 34, 32)
    pdf.rect(0, PH - 14, PW, 14, "F")
    pdf.set_font("Helvetica", "B", 9)
    pdf.set_text_color(*GOLD)
    pdf.set_xy(MARGIN, PH - 10)
    pdf.cell(60, 6, "FORMO  RENOVATION", align="L")
    pdf.set_font("Helvetica", "", 7)
    pdf.set_text_color(*MUTED)
    pdf.set_xy(PW/2 - 40, PH - 10)
    pdf.cell(80, 6, f"{project}  ·  {address}", align="C")
    pdf.set_xy(PW - MARGIN - 40, PH - 10)
    pdf.cell(40, 6, date, align="R")


# ── Routes ──────────────────────────────────────────────────────────────

@app.route("/")
def index():
    return render_template("index.html")


@app.route("/generate", methods=["POST"])
def generate():
    if "pdf" not in request.files:
        return jsonify({"error": "No PDF uploaded"}), 400

    pdf_file = request.files["pdf"]
```

```
mode = request.form.get("mode", "client")
include_descriptions = (mode == "internal") and bool(ANTHROPIC_KEY)

with tempfile.TemporaryDirectory() as tmp:
    pdf_path = os.path.join(tmp, secure_filename(pdf_file.filename or "report
    pdf_file.save(pdf_path)

    meta = extract_meta(pdf_path)

    try:
        imgs = extract_images(pdf_path, tmp)
    except Exception as e:
        return jsonify({"error": f"Could not extract images: {str(e)}"}), 500

    if len(imgs) < 2:
        return jsonify({"error": "No photos found in this PDF."}), 400

    cover_img  = imgs[0]
    photo_imgs = imgs[1:]

    # Build captions
    captions = []
    for i, img_path in enumerate(photo_imgs, 1):
        if include_descriptions:
            cap = ai_describe(img_path, i)
        else:
            cap = {"section": "General", "title": f"Photo {i}",
                   "description": "", "tags": []}
        captions.append(cap)

    # Build PDF
    out_pdf = os.path.join(tmp, "report.pdf")
    try:
        build_pdf(meta, cover_img, photo_imgs, captions, out_pdf)
    except Exception as e:
        return jsonify({"error": f"PDF generation failed: {str(e)}"}), 500

    if not os.path.exists(out_pdf):
        return jsonify({"error": "PDF was not created"}), 500

    addr = meta.get("address", "Report").replace(" ", "_")
    suffix = "_Internal" if include_descriptions else "_Client"
    filename = f"Formo_{addr}{suffix}.pdf"

    return send_file(
        out_pdf,
        mimetype="application/pdf",
```

```
            as_attachment=True,
            download_name=filename
        )


if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=False)
```